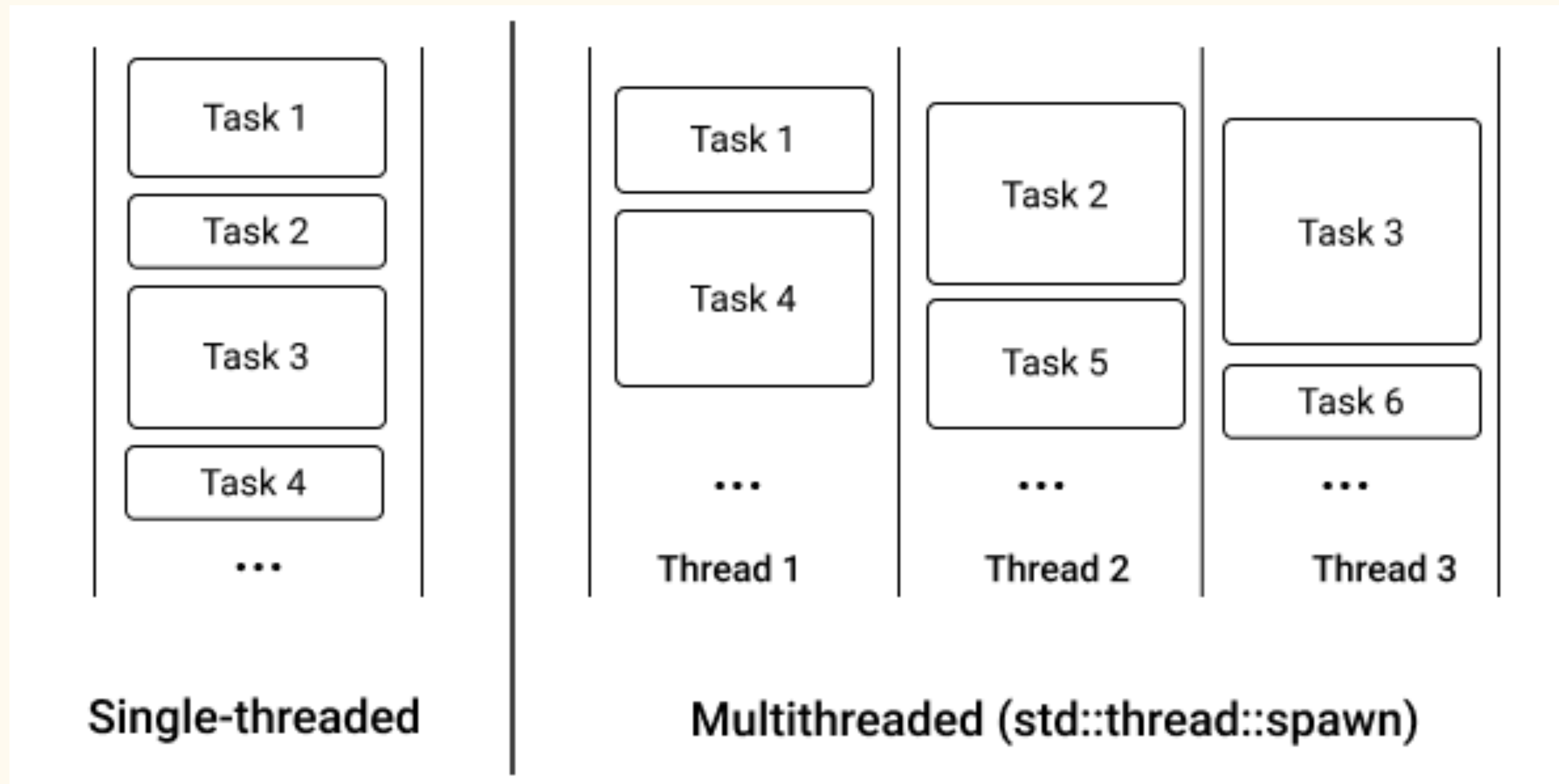


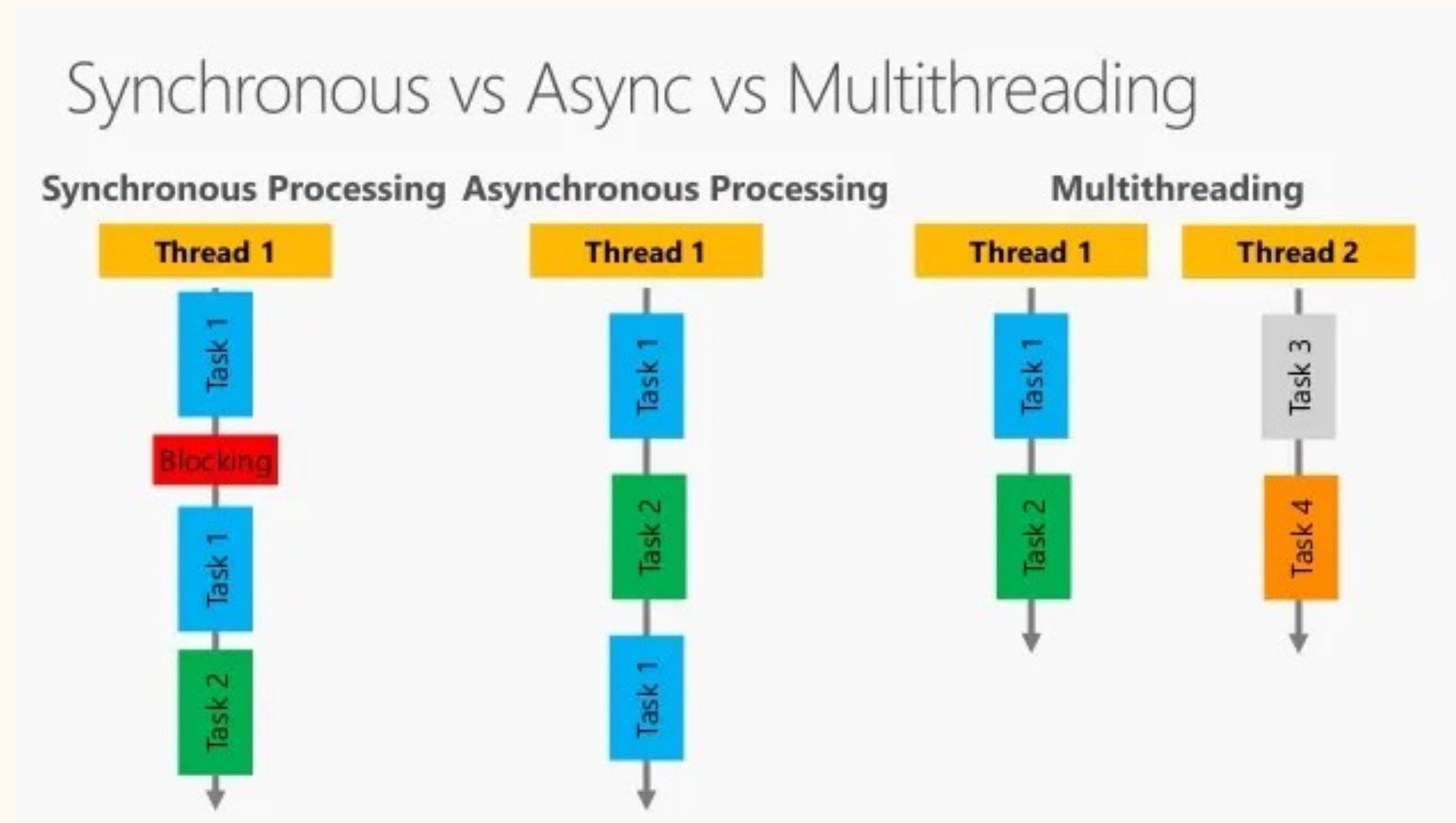
MULTI THREADING

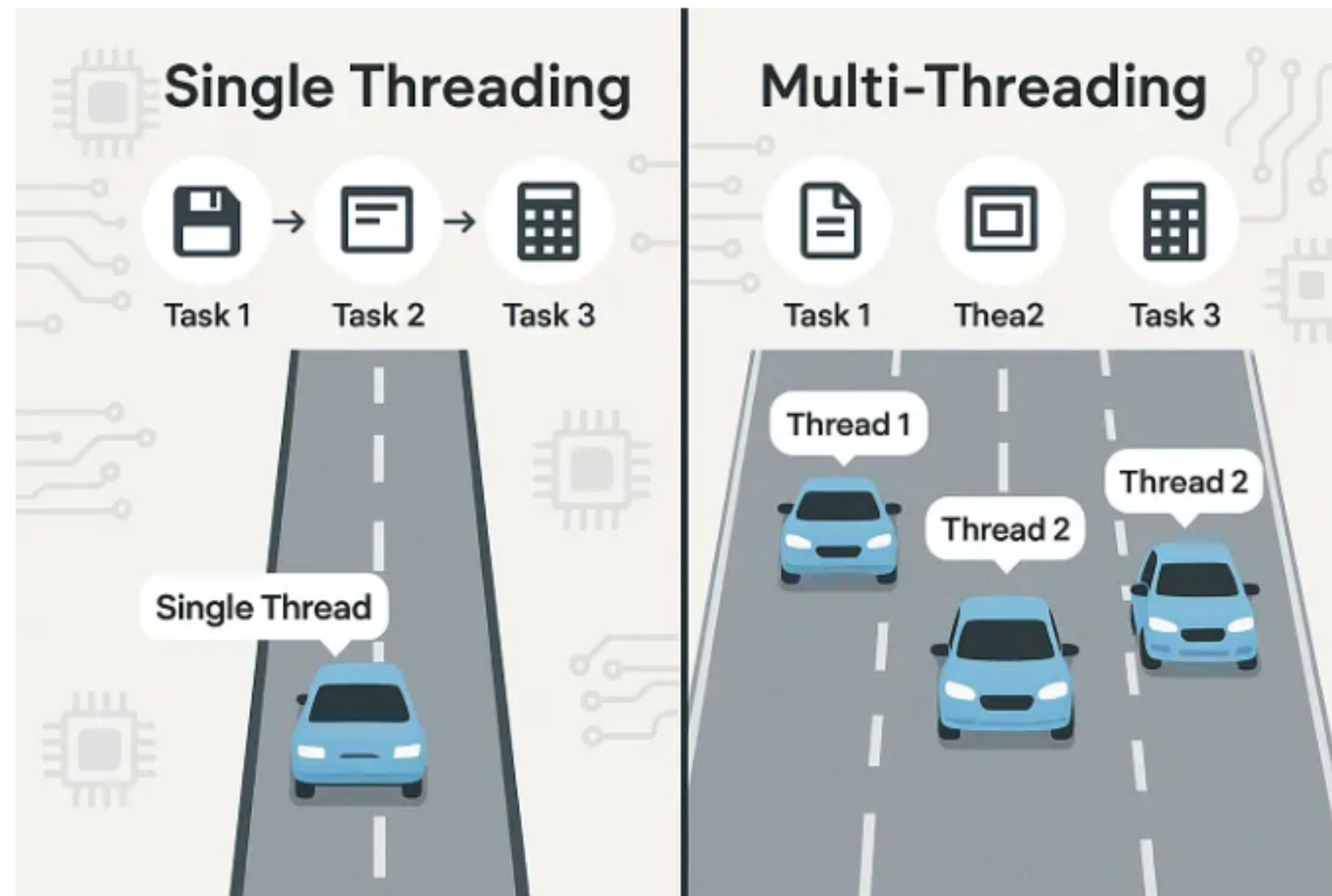
I. Lesson

Single thread vs Multi thread



Single thread vs Multi thread vs asynchronous





In computing, **threads** are like **mini-tasks** running within a program. A thread is the smallest unit of execution, and how a program handles multiple tasks depends on whether it's **single-threaded** or **multi-threaded**. Let's break it down in simple terms.

Single Threading: pros and cons

✓ Advantages of Single Threading:

- Easier to develop — No need to manage multiple threads.
- Consumes less memory — Uses fewer system resources.
- Predictable execution order — Tasks are completed sequentially, reducing complexity.

✗ Disadvantages of Single-Threading:

- Slower performance — One task at a time means no parallel execution.
- Not ideal for complex applications — The UI may freeze while processing intensive tasks.
- Inefficient CPU usage — Wastes processing power when waiting for tasks to complete.

📌 **Example:** A basic text editor that saves a document. While saving, the application may become unresponsive because it handles only one task at a time.

Multi Threading: pros and cons

What is Multithreading?

A multi-threaded application can execute multiple tasks simultaneously by splitting processes into separate threads, which improves performance and responsiveness.

✅ Advantages of Multi-Threading:

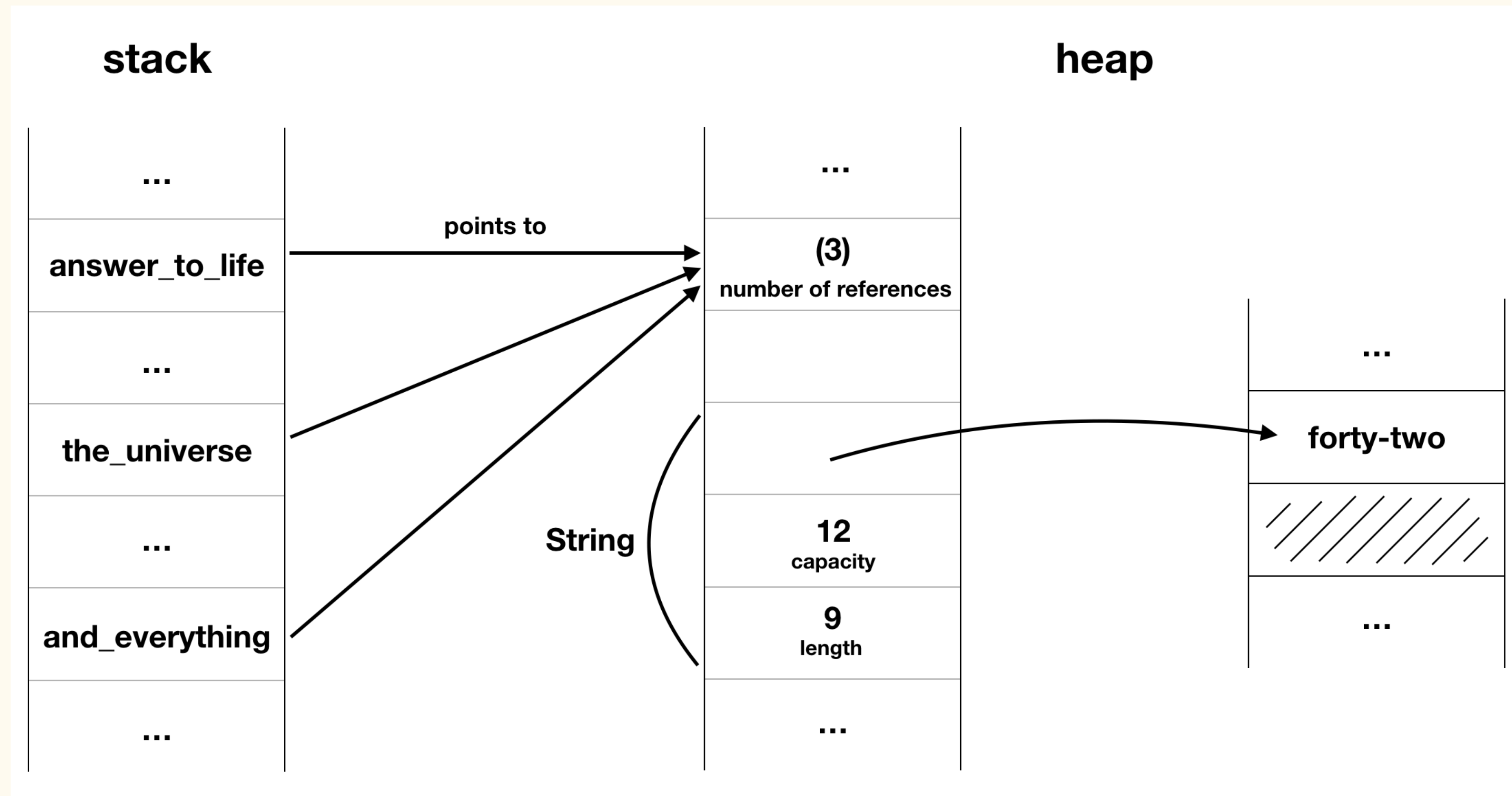
- **Improved performance** — Uses multiple CPU cores efficiently.
- **Better user experience** — The UI remains responsive while background tasks run.
- **Enables multitasking** — Handles multiple operations at once, increasing efficiency.

❌ Disadvantages of Multi-Threading:

- **More complex to develop** — Requires careful thread management and synchronization.
- **Potential for bugs** — Issues like race conditions and deadlocks can arise.
- **Increased memory usage** — More threads consume more system resources.

📌 **Example:** A modern web browser that loads multiple tabs simultaneously or a video editing application where rendering happens in the background while the user continues editing.

Reference counting (Rc<String>)



```
use std::rc::Rc;

// Rust can infer all these types; written out for clarity
let s: Rc<String> = Rc::new("shirataki".to_string());
let t: Rc<String> = s.clone();
let u: Rc<String> = s.clone();
```

For any type T , an $Rc<T>$ value is a pointer to a heap-allocated T that has had a reference count affixed to it. Cloning an $Rc<T>$ value does not copy the T ; instead, it simply creates another pointer to it, and increments the reference count. So the preceding code produces the situation illustrated in Figure 4-12 in memory.

Each of the three $Rc<String>$ pointers is referring to the same block of memory, which holds a reference count and space for the `String`. The usual ownership rules apply to the `Rc` pointers themselves, and when the last extant `Rc` is dropped, Rust drops the `String` as well.

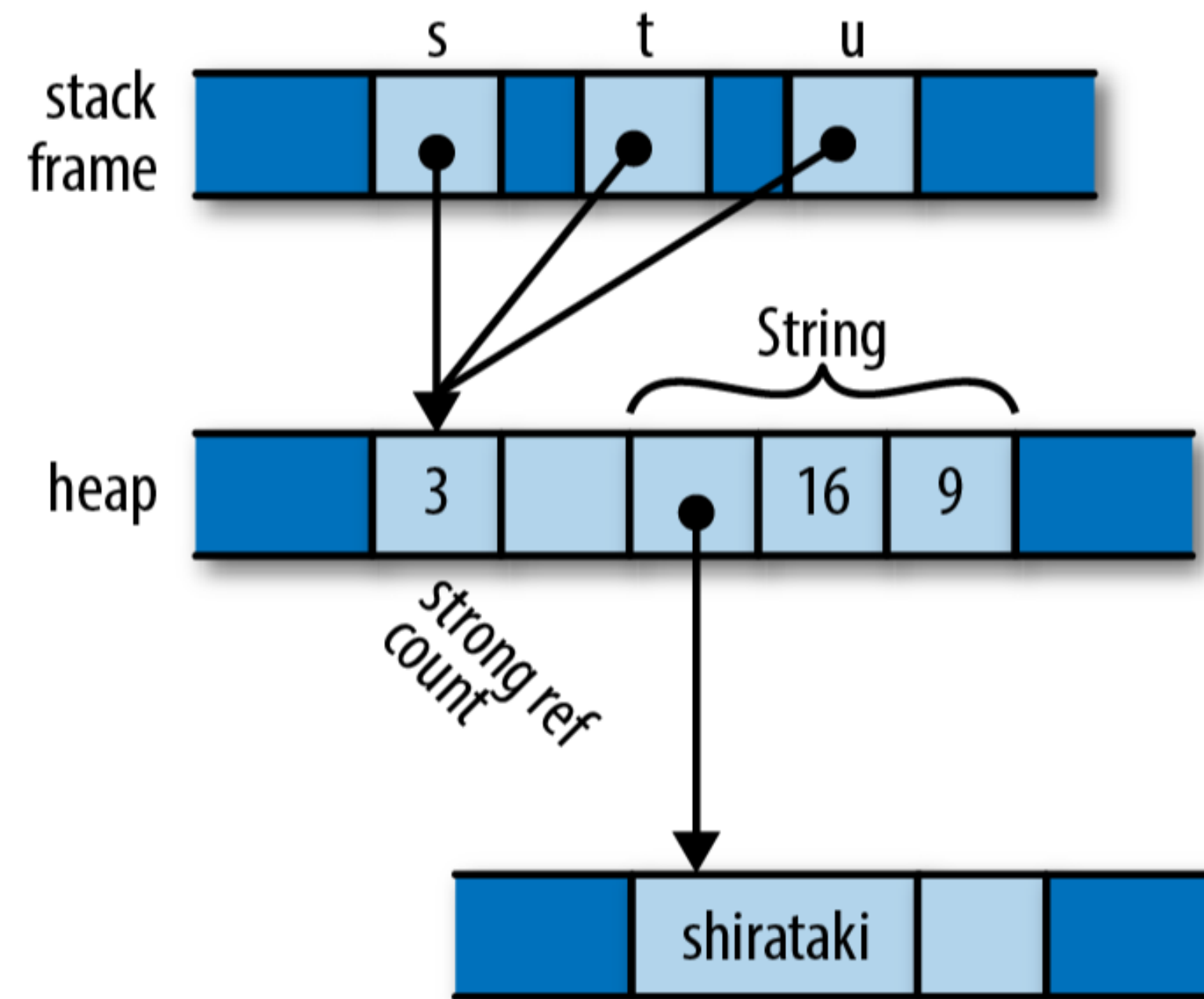
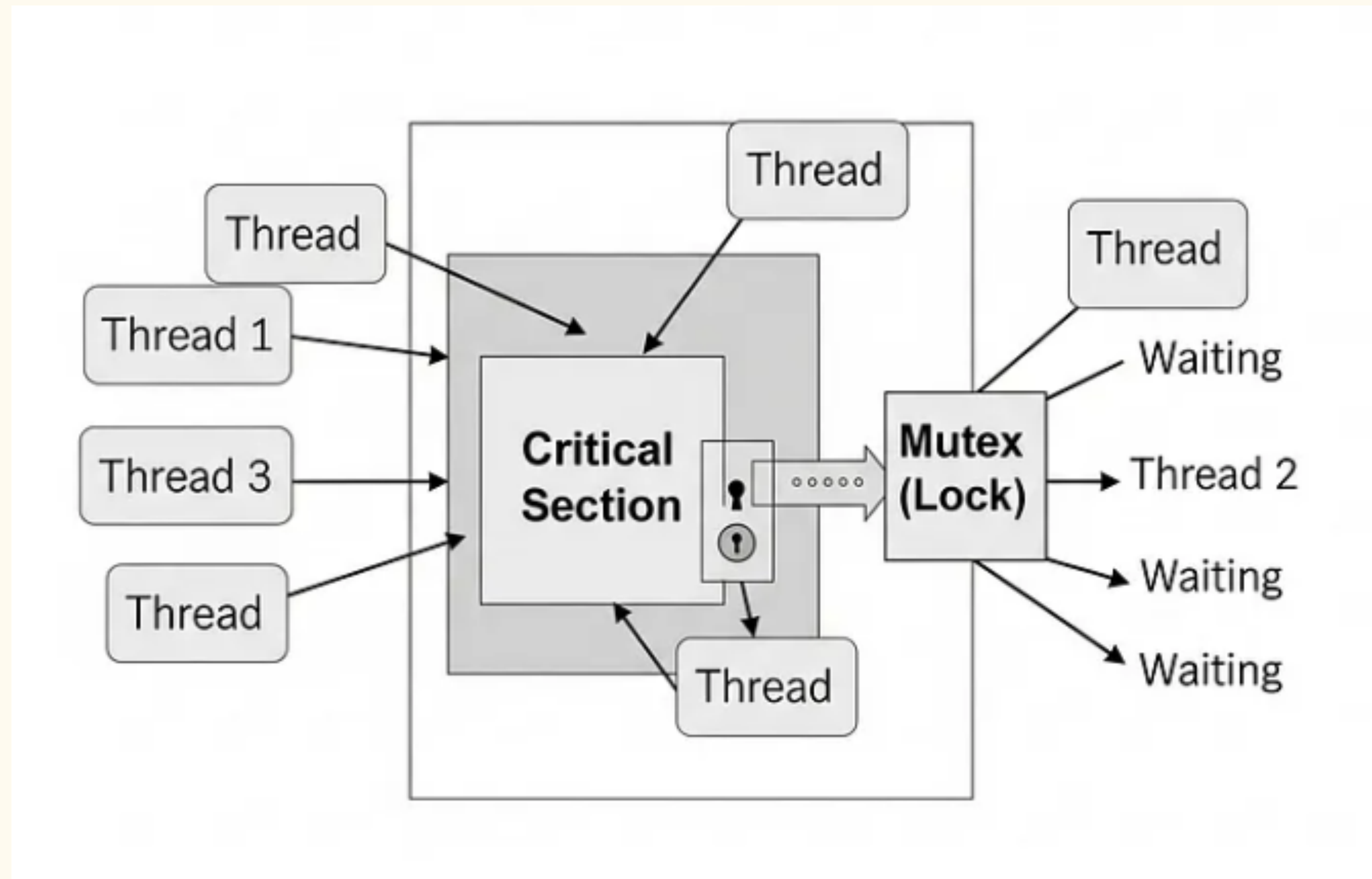
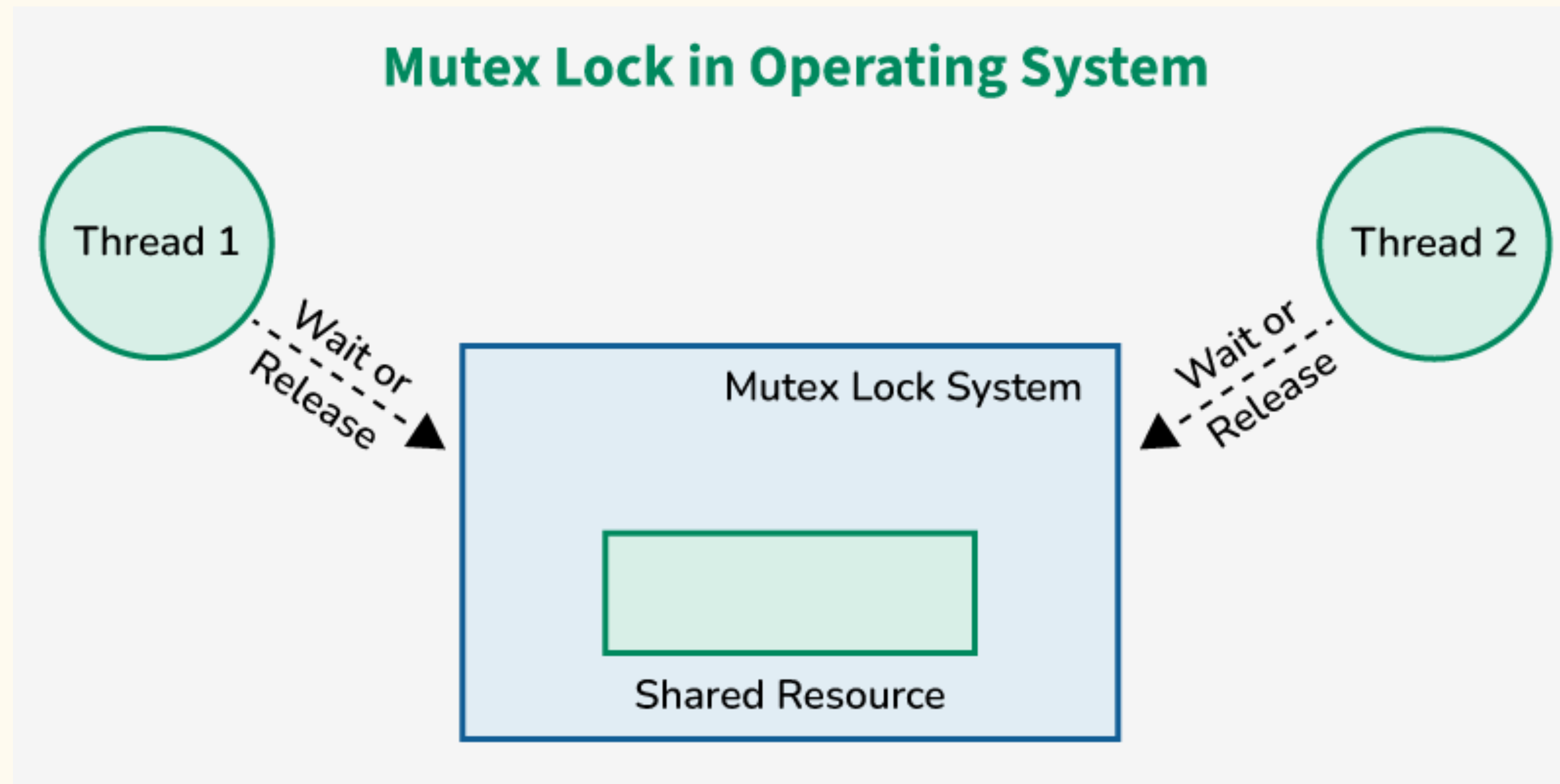


Figure 4-12. A reference-counted string, with three references

Mutex & Locking



Mutex & Locking



II. Project



Rc::strong_count

```
println!("Original balloon: {}", balloon);
println!("Reference count: {}\n", Rc::strong_count(&balloon));

// Share the balloon with your sister
let sister_balloon = Rc::clone(&balloon);
println!("Sister now has the balloon: {}", sister_balloon);
println!("Reference count: {}\n", Rc::strong_count(&balloon));
```

When to use RC ?

When to Use Rc? 🤔

Use Rc when:

- You want to share a toy (data) with friends
- Nobody needs to change the toy
- Everyone is in the same room (thread)

Threads

```
let handle1 = thread::spawn(|| {  
    println!("📄 Downloading file... (5 seconds)");  
    thread::sleep(Duration::from_secs(5));  
    println!("✅ Download complete!");  
});
```

```
// Wait for all threads to finish  
handle1.join().unwrap();  
handle2.join().unwrap();  
handle3.join().unwrap();
```



Mutex with Teddy Bear



```
fn main() {
    println!("🧸 Kids Sharing a Toy Box!\n");

    // Create our special toy box with a teddy bear
    let toy_box = Mutex::new("Teddy Bear");

    // Child 1 wants to play
    {
        let mut my_turn = toy_box.lock().unwrap();
        println!("👧 Child 1: Yay! I got the {}!", my_turn);
        *my_turn = "Happy Teddy Bear"; // Child 1 makes the teddy happy
        println!("👧 Child 1: I made it a {}!", my_turn);
    } // my_turn ends here - toy goes back in box!

    println!("📦 Toy is back in the box!\n");

    // Child 2's turn
    {
        let my_turn = toy_box.lock().unwrap();
        println!("👦 Child 2: Look! The teddy is {}!", my_turn);
    } // Toy goes back again

    println!("📦 Toy is back in the box again!");
    println!("\\n✅ Everyone took turns nicely!");
}
```



Arc Mutex with Teddy Bear



```
fn main() {
    println!("👶 Multiple Kids Sharing a Toy Box!\n");

    // Create a toy box that can be shared between kids (threads)
    let toy_box = Arc::new(Mutex::new("Teddy Bear"));
    let mut handles = vec![];

    // Each kid wants to play
    for kid_number in 1..=3 {
        let my_toy_box = Arc::clone(&toy_box);

        let handle = thread::spawn(move || {
            // Kid tries to get the toy
            let mut my_turn = my_toy_box.lock().unwrap();

            println!("👶 Kid {}: Yay! I got the {}!", kid_number, my_turn);

            // Kid plays with toy for a while
            thread::sleep(Duration::from_secs(1));

            // Kid makes the toy happy
            *my_turn = "Happy Teddy Bear";

            println!("👶 Kid {}: I made it a {}!", kid_number, my_turn);

            // When my_turn is dropped here, the toy goes back in the box!
        });

        handles.push(handle);
    }

    // Wait for all kids to finish playing
    for handle in handles {
        handle.join().unwrap();
    }

    println!("\n✅ All kids finished playing!");
}
```


Let's code!

Your turn!